ECE 350: Real-Time Operating Systems Winter 2027

Lecture 26 — Solid State Drives and File Systems

Jeff Zarnett *2026-08-14*

Solid Snake? No, Solid State

Just as the previous topic started off with talking about the physical construction of a magnetic hard drive, let's start by talking about how solid state drives work. You might have even held in your hands a hard drive that died on me in 2014, to allow you to explore its mechanical components. The SSD version of this does not involve quite as much hands-on experience with the device, for two reasons. For one, I don't have a dead SSD device to show you. The other is that it's much less interesting when you hold it in your hands. It looks like a RAM stick, honestly.

Once again, solid state drives are called that because they have no moving parts. That means the question is not the scheduling algorithm for reads and writes; it's a uniform access time device and therefore it can be FCFS or perhaps that with some variation to reflect priorities. And yet, SSDs have some nuances to them that make this more interesting than "hard drives, but faster and uniform".

SSDs are typically built using NAND Flash memory. The details of that are interesting, but perhaps too low-level for us to think about. The important thing that matters for the discussion here, however, is that SSDs are write-limited devices. That is, they have a maximum number of write cycles and after that, they're finished. Should I have said cooked? They're cooked.

This is profoundly weird as compared to everything else in the system. Yes, we understand that nothing is forever, or things can break or get damaged. But for the other components of the system (CPU, RAM, magnetic drive, etc.) there's no maximum number of things. Can you imagine? Sorry, your CPU has reached its instruction execution maximum and cannot do any more, guess you have to buy a new one. Wait. I'm wrong. Printers from predatory companies do this and I *hate* it. Don't buy a printer that does this. The toner cartridge is finished when it is completely empty, not when HP (yes, I'm naming names) has decided you should have to buy a new one. In perhaps a less infuriating example, Lithium-Ion batteries also have a lifecycle that's measured in the number of discharge cycles. And that is a limitation imposed by the chemistry of the battery, not arbitrary manufacturer "shareholder value" calculations.

Why do the NAND flash chips break down with writing? It's the physics of the thing: the high field and current stress from writing to the chip breaks down the oxide which eventually renders a given cell useless [Kha09]. Once that has happened to enough cells, the device is no longer capable of writing data. Maybe you can read it. The device is rated for N cycles for each of the cells, but exactly how many depends on the specific drive and its technology. Under normal use, you will probably never encounter this: your laptop, phone, or other device will likely be retired for other reasons before you ever get close to exhausting the ability to write to disk.

How big is N cycles, though? It's typically something like 10 000 cycles for an individual cell [ADAD23]. You might think that's a lot or you might think it's very little. Let's say it's your laptop. Lots of files on the laptop are written once or rarely. Even these lecture notes – yes, I write them and I revise them from time to time but once the course material is written and all the typos have been fixed (haha, as if), will it change again? Not for a long time. But we learned about the swap file earlier: the swap file is read and written a lot! 10 000 writes to the swap file can happen in a day, easily, so then what?

This has led us to something important: when the SSD writes to the same file, that data might not be in the same place(s) on disk anymore. In fact, even if we wanted to, it would be slow: to write data to a particular block, we have to erase what's there and then write the new block. But erasing is slow. So the solution is to write to a new location and then we can go back and erase the old location at a later date. This has some similarities with page faults: if there is an empty frame in memory, it means we can do the read into that frame immediately and evict another page from a frame later, asynchronously.

The consequences of this idea that physical locations move sometimes immediately leads to four things that matter in the implementation: (1) there has to be some indirection so when the OS says "read block 9876" we map that to a physical location; (2) some sort of wear-levelling to distribute the writes to preserve write-capacity, (3) a form of garbage collection, and (4) there are additional writes that are overhead. There's more to SSDs than those things, but let's do these and come back to the rest later.

Flash Translation Layer. The level of indirection is called FTL (Flash Translation Layer, not Faster Than Light). This layer needs to take a request and determine its physical location. We will examine a few different strategies for this.

Wear-Levelling. Given that every block is as fast as any other, it truly does not matter from the perspective of access speed where the data is. That means that it is possible for the device to always choose to write things to the available area that has the fewest number of write cycles accumulated.

Recognizing that a lot of files don't change very often, it's possible that you have some areas of the disk that are allocated but written to only once. A thorough wear-levelling program would move those things around periodically to ensure that we are really wearing things down as evenly as possible.

Garbage Collection. Just like in languages that do this for memory, garbage collection is a periodic process of cleaning up things that are no longer referenced/needed. The process requires a way of keeping track of what is needed and what is not; with that information, the device can iterate over the pages that we have and reclaim the space to make it available again. A more complex algorithm can reduce the amount of work done, but the basic idea does not change. Part of the motivation for doing it with garbage collection rather than disposing of the data immediately is that the garbage collection process can run at any convenient (i.e., low-utilization) time rather than synchronously when the drive is under heavy use.

Write Amplification. Write Amplification is the term that is used to express how much additional writing takes place above the logical write that the OS or application asked for. For example, if the write is a 4 KB chunk of a file, the bytes written within the flash varies. How much extra depends on the workload and the hardware of the SSD device but it can be 1.2 - 3.0 times (so a 4 KB write results in, on average, 4.8 to 12 KB changed on the drive) [HEH+09]. Knowing that every write is consuming some of the limited lifespan of the chips, it almost goes without saying that write amplification is undesirable and something that we'll aim to minimize.

FTL Organization

Before we get into the details here, it's important to note that the word "block" is used to refer to a unit of physical storage and does not necessarily have the same size as a page. This does mean we sometimes have device characteristics like what Wikipedia shows where a 4 KB page is the logical unit for a page but if we do an erase operation we end up erasing 256 KB. Exact numbers will, of course, vary based on the device purchased.

The simplest, and less-than-ideal, approach to FTL mapping is directly mapped: a read of logical page N is mapped directly to a physical page N and a write is done by reading the whole block with the page in it, erase it, and then overwrite that location with the new version [ADAD23]. From what we know already, it is clear why this is bad: it's slow to put the erase action on the critical path for writing the data, and it also wears out that particular location faster. So this approach is clearly not going to do it.

The current approach used by SSDs is called *log-structured*. This means that when we write a logical block N, it just gets added to the next free space in the currently-being-written block [ADAD23]. This requires, obviously, some mapping to exist that translates a logical location to its physical one. The mapping has to be stored in stable storage, otherwise powering the system down means losing all information about what is where, which would be contrary to the idea of the device as persistent storage.

Still, the mapping table is potentially a large amount of data; for every 1 TB of storage where pages are 4 KB, it costs 1 GB of storage just for the mapping table [ADAD23]. That actually does not seem so bad in some absolute

sense – 1/1000 of the storage for administration seems like a pretty reasonable amount of overhead. The linked source considers this to be a huge amount of overhead and goes through a couple of techniques to reduce the amount of entries in this table; but it is not that interesting so we will skip it.

This approach does lead to garbage, in the sense of garbage collection mentioned previously. When writes are treated as append, there is a need to go back and clean up. If a whole block is no-longer valid data, then it is simple to erase it and consider it free. If a part of the block is still needed, though, it would be a problem to erase the whole thing. And yet, we would not be happy with the idea of wasting 252 KB just because one valid page remains in the block. The solution to that is a sort of compaction: if there is garbage in this block, write the valid sections into a new location and then the whole block can be erased.

Moving things around to make the data compact does help with wear-levelling by ensuring that data can move to new locations periodically, but it also results in more writes to the device that consume its limited lifespan...

TRIM

All of this discussion might have led to the idea that this is all internal to the SSD device itself, and of little concern to the operating system. That's not quite true, because of the existence of the `TRIM` command. Its purpose is for the operating system to inform the SSD what data is no longer needed and can be cleaned up.

The command is needed because many file systems work similar to C's approach to deallocation of memory: when a file is deleted, it is just marked as no-longer in use but the actual data is not explicitly deleted/cleared. So the metadata about the files is updated, but the SSD has never been informed that the data of that file on the disk is garbage now. Remember, this didn't matter for HDDs because if the OS knows that block 9876 is free, it can directly issue a write to that location and the HDD does not track or care about which parts of the storage medium are used.

There is a design decision as to when to issue the `TRIM` command. The Red Hat Enterprise Linux docs[1] say there are three options for this: online (continuous), batch, and periodic. The names explain pretty much what they do: the online version triggers the command immediately when a file is deleted; batch happens on-demand and cleans up all the things that are pending; and periodic of course means it runs regularly on a schedule.

The potential disadvantage of the continuous approach is that it does make this operation synchronous and it can lead to latency spikes on the system. The Arch Linux docs[2] reference the idea that the problem has to do with how the TRIM command was implemented in the hardware standard. No need for us to get into the details there, but let's leave it that periodic is the right choice in most circumstances.

This discussion about TRIM leads us very nicely into the next relevant topic: how do file systems work? In a previous course, we will have covered the interface point of view.

File System Implementation

Now it is time to go behind the scenes of how the file system lives up to the interface we learned about in the previous course. The implementation is somewhat complicated, and to keep the size of the problem manageable we will worry about storing files on hard disks. Hard disks themselves make a good choice for this: they are sufficiently large and sufficiently cheap, to start with, to store the data that we want to store. We can read an arbitrary part of the disk. Finally, we can write to the same part of disk as many times as we want (disks don't, at least on the time frames that we are concerned about, wear out). Recall also that disks operate on blocks which, in their physical representation, comprise one or more sectors.

Let us take a look at the layers of the file system's design, from [SGG13].

[1] `https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/8/html/managing_file_systems/discarding-unused-blocks_managing-file-systems`

[2] `https://wiki.archlinux.org/title/Solid_state_drive`

The File System. The file system user interface is intended for the convenience of the user and for application programmers. This is the level we have just examined in the previous topic; the more interesting part is what happens when we need to fulfill the promises that the file system makes.

I/O Control. At the I/O control level, we are dealing with device drivers and interrupt handlers to transfer data. The inputs are fairly high-level commands, along the lines of "read block 1234". Its outputs are hardware specific-instructions to the hardware controller. Usually this is by writing bit patterns in the I/O controller memory.

Basic File System. Despite the awful name, this is the level at which we start to deal with physical blocks on the disk. A physical block is identified by its numerical physical address: drive 0, cylinder 12, track 7, sector 1. This layer is also responsible for buffers and caches that are used to hold various commonly-accessed regions (e.g., the temporary directory). Yes, caching and buffers can appear in the hard disk as well, with performance improvements traded off against the risk of data loss if power is suddenly cut.

File Organization Module. This module is aware of files and their logical and physical blocks. This translates a logical block address to a physical block address and keeps track of free space (unallocated blocks).

The Logical File System. This last level is for managing metadata: file system structure, directory structure, and maintaining the file structure. File metadata is maintained in a *file control block* (FCB). The UNIX term for this is *inode* and it is the place where file info is stored: ownership, permissions, locations of the file contents.

Disk Organization

Although there are a million different file systems (UFS, HFS+, ZFS, NTFS, ext3, FAT32...) that are all significantly different, there are some general principles that we can examine. A file system will need to keep track of the total number of blocks, the number and locations of free blocks, the directory structure, and the files themselves.

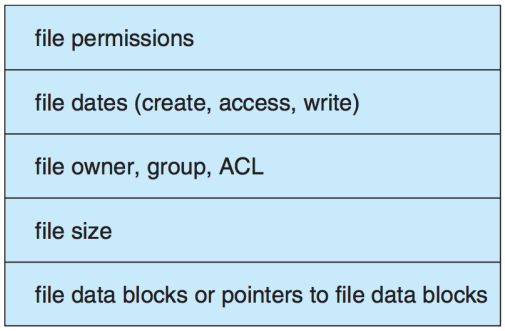
On at least one disk somewhere in the system, there will need to be some information about how to boot up the operating system. Not every disk has the operating system on it, but if there is an OS the boot loader is usually put in the first block. When the power button is pressed on the case, the BIOS starts up and transfers control to whatever is found at that first block (which is hopefully your boot loader that launches the OS, or gives you the option of which OS to start).

Disks may be split, logically, into several different areas, or *partitions*. Accordingly, there will be a partition table that indicates what part of the disk belongs to which partition. In the Windows world we often see the whole disk is in one logical partition (the C: drive, for example, taking the whole primary disk). In Linux we often see the disk divided up to have partitions for different things: temporary/swap directory, home directories, boot partition, et cetera...

There are several structures that are likely to be in memory for performance reasons [SGG13]:

1. **Mount Table**: Information about each mounted volume (disk/partition).
2. **Cache**: Directory info for recently accessed directories.
3. **Global Open File Table**: Copy of the FCB for each open file.
4. **Process Open File Table**: References to the global open file table, sorted by process.
5. **Buffers**: places where data read from or to be written to disk resides after or before the actual disk operation.

Creating a new file is the job of the logical file system: allocation of a new FCB (or re-use of an existing free FCB). A typical FCB might look like this:



File Control Block (FCB) example [SGG13].

If a user (application) actually wants to make use of a file, the open system call is needed. Open operates on names, the user comprehensible version. The file system must then check the global open file table to see if the file is already open somewhere in the system. If so, if that file is opened for exclusive access, then the open routine returns with an error. If it is open but for non-exclusive access, then there is no need to search for the file or retrieve it by directory; we just make another reference in the process open file table. If the file is not already open, then it needs to be retrieved; once the file is found, the FCB is copied into the global open file table and the appropriate reference is added to the process table [SGG13].

The process open file table can contain some additional information, like the next section to read or write, the access mode when the file is open, and so on. The open system call returns a pointer to the file table and this is the route through which the application performs all file operations. In UNIX this reference is called a *file descriptor*; in Windows it is called a *file handle* [SGG13].

The opposite operation to opening a file is obviously to close it. When a process closes a file, the entry from the process open file table can be removed. If this is the last reference to the file a process has, then the file can also be removed from the global open file table. Metadata may be updated on close.

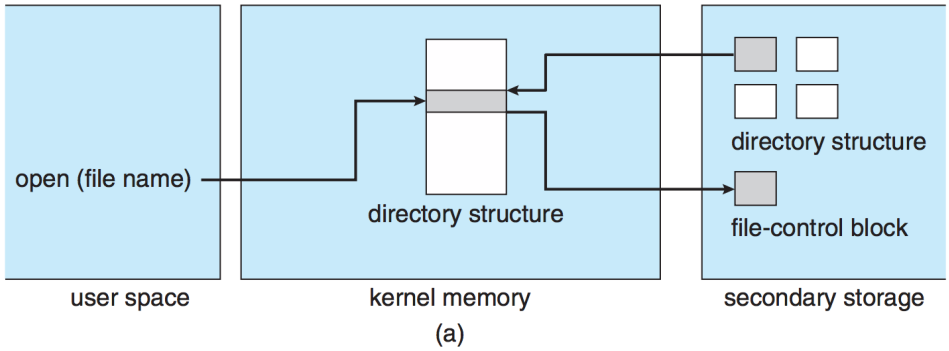
Searching with Spotlight. A small aside on the subject of metadata.

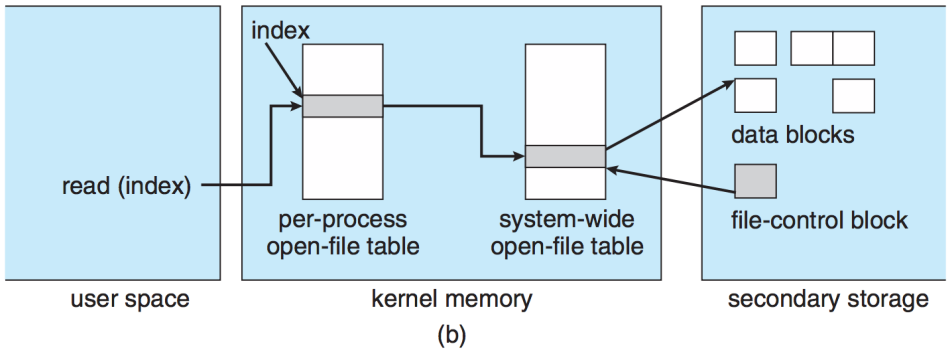
An example from Mac OS X: the Spotlight system-wide desktop search. Spotlight was introduced quite some time ago but it was a revelation compared to the previous search options that were slow and did not necessarily include the file content, just the file names. Spotlight runs queries quickly over the metadata of the system. Spotlight examines each file on creation and modification and uses it to update the metadata related to that file. Those items are then added to the Spotlight index which can be queried very rapidly [Sir05].

This is a specific case of preparing the data in advance to make searching or exporting operations fast. Another example is in exporting data to Excel. To export this data, there are two possible ways to do it. One is to examine and process the data for the Excel file at the time of the user request. The other is to maintain that data separately, and when the Excel report is asked for, take that data and put it in the Excel file.

Part of the difficulty with maintaining metadata is when to update it. The longer the time between metadata updates, the higher the chance that there will be a user request that gets back out-of-date data. The faster the metadata updates, the more time and processing power is spent creating and updating this metadata.

The diagram below shows how a file is opened and how a read takes place:





File System Structures for (a) file open and (b) file read [SGG13].

References

[ADAD23] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 1.10 edition, November 2023.

[HEH+09] Xiao-Yu Hu, Evangelos Eleftheriou, Robert Haas, Ilias Iliadis, and Roman Pletka. Write amplification analysis in flash-based solid state drives. In *Proceedings of SYSTOR 2009: The Israeli Experimental Systems Conference*, SYSTOR '09, New York, NY, USA, 2009. Association for Computing Machinery.

[Kha09] Faraz I. Khan. Endurance characterization and improvement of floating gate semiconductor memory devices. 2009.

[SGG13] Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. *Operating System Concepts (9th Edition)*. John Wiley & Sons, 2013.

[Sir05] John Siracusa. Mac OS X 10.4 Tiger, 2005. Online; accessed 24-May-2015. URL: `http://arstechnica.com/apple/2005/04/macosx-10-4/`.